Local ZetaERP — run a screen locally to verify it before a PR merge

Audience: any contributor who changed a screen/service and needs to prove it actually works before the PR is merged. **Scope:** standing up a **local, throwaway** ZetaERP on your own PC so you can open the screen in a browser. This is the "integration check" that a passing offline script or a clean-looking diff **cannot** give you.

> Read `../../CONTRIBUTING.md` and `../../SECURITY.md` first — their non-negotiables override anything here. Deploying to **production** is a different, owner-only process: see `../SOP-commit-and-deploy.md`. This runbook never touches production.

Why this exists

A Moqui screen (`*.xml` with FreeMarker + Groovy) is **server-rendered** — a running Moqui turns it into HTML. So "does my change work?" is only answerable by loading the code into a real Moqui and opening the page. Opening the `.xml` file, or running a mocked offline Groovy check (one that stubs `ec` and runs the screen's logic + FreeMarker template), verifies **logic and template syntax** but not the real render: the `/apps` and `/qapps` shells, the stylesheet/JS loading, real entity data, screen transitions, or how it looks. Those are exactly where "works on paper / breaks in the browser" bugs hide. A draft PR whose description says "*integration checks remain open*" means **this step was skipped** — close that gap here before marking the PR ready.

What you get — and what you don't

- ✅ A faithful local copy of the running app (same image, same framework patches) to render and click through any `/apps`, `/qapps`, or `/vapps` screen.
- ❌ **Not** for real transactions, real printing (QZ signing is disabled locally), or real tenant data. It is a disposable verification rig.

Security — read before you start

These are **constraint #1 and #2** from `CLAUDE.md`, applied to your laptop:

1. **Never copy production data to your machine.** The prod database holds real tenant business and legal data (GSTINs, prices, stock) and user/security data. Do **not** restore the prod `pgdata` volume or a raw `pg_dump` of prod. Use a **sanitized** snapshot from the owner, or Moqui demo data (Step 3).
2. **Never put a production secret in a local file or a commit.** The local compose below uses throwaway passwords you choose. The real prod DB/OpenSearch passwords and the QZ signing key stay on the prod host only. **Do not commit** `docker-compose.local.yml` — add it to your local `.gitignore` (it is already ignored if it matches `*.local.yml`; confirm with `git check-ignore`).

Prerequisites

- **Docker Desktop** (or Docker Engine + Compose v2) with ~**5 GB** free RAM for the container.
- **Git**, and this repo cloned (`git clone https://github.com/amalnathkp13-boop/zenith-erp.git`).
- No prod/SSH access is required for Path A below.

Step 1 — get the app image

The app image (`zetaerp-moqui:latest` , ~530 MB compressed) is **not in a public registry**. Two ways:

Path A — load a prebuilt image from the owner *(recommended — fastest, no secrets to handle)*

The owner exports it once from the running container and shares the file (internal channel, not a public link):

```
# owner runs on the prod host:
docker save zetaerp-moqui:latest | gzip > zetaerp-moqui.tar.gz
```

You load it locally and re-tag it for local use:

```
docker load < zetaerp-moqui.tar.gz
docker tag zetaerp-moqui:latest zetaerp-moqui:local
```

Path B — build it from source *(for maintainers, or if you can't get the tar)*

The build recipe is `deploy/Dockerfile` : it starts from the **public** pinned base `ghcr.io/moqui/moqui-framework` (no login needed) and overlays the component tree + framework patches. It expects a build context containing `zenith-erp/` (the component tree), `framework-patches/` , `zenith-classes/` , a `MoquiDevConf.xml` , and a `qz-private-key.pem` .

- The component tree and patches come from this repo (`component/` → `zenith-erp/` , plus `docs/infra/framework-patches/` and `docs/infra/zenith-classes/`). For an exact reproduction of the **live** component tree, the owner runs `sync-build-context.sh` first.
- `qz-private-key.pem` is a **secret** used only for receipt signing — **not needed to verify a screen.** For a local build, use a throwaway placeholder so the `COPY` succeeds: `openssl genrsa -out qz-private-key.pem 2048` (local-only; never commit it). Receipt printing won't work locally, which is irrelevant for screen verification.
- Build: `docker build -t zetaerp-moqui:local -f docs/infra/deploy/Dockerfile <build-context-dir>`

> Building is slower and needs the context assembled; Path A is the normal choice for a teammate who just wants to verify a screen. Ask the owner for the tar.

Step 2 — the local compose file

Save this as `docker-compose.local.yml` (do **not** commit it). It mirrors the production topology (Postgres + OpenSearch + Moqui) with **local-only** passwords and **fresh** volumes.

```
# docker-compose.local.yml  —  LOCAL VERIFICATION ONLY. Throwaway passwords + empty volumes.
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: moqui
      POSTGRES_USER: moqui
      POSTGRES_PASSWORD: CHANGE_ME_local   # any value; must match moqui's
entity_ds_password below
      TZ: Asia/Kolkata
    volumes:
      - localpgdata:/var/lib/postgresql/data   # LOCAL, starts empty — never the prod volume
    mem_limit: 1g

  opensearch:            # only needed for search-backed screens; heaviest service — omit if
tight on RAM
    image: opensearchproject/opensearch:2.15.0
    environment:
      - discovery.type=single-node
      - plugins.security.disabled=true
      - OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m
      - OPENSEARCH_INITIAL_ADMIN_PASSWORD=CHANGE_ME_local_os   # local only
    ulimits:
      memlock: { soft: -1, hard: -1 }
    volumes:
      - localosdata:/usr/share/opensearch/data
    mem_limit: 1536m

  moqui:
    image: zetaerp-moqui:local          # from Step 1 (Path A load, or Path B build)
    depends_on: [db, opensearch]
    environment:
      - TZ=Asia/Kolkata
      - entity_ds_db_conf=postgres
      - entity_ds_host=db
      - entity_ds_port=5432
      - entity_ds_database=moqui
      - entity_ds_user=moqui
      - entity_ds_password=CHANGE_ME_local   # must equal db POSTGRES_PASSWORD
      - elasticsearch_url=http://opensearch:9200
      - JAVA_TOOL_OPTIONS=-Xms1g -Xmx2g -Dbitronix.tm.serverId=zetaerp-local -
Duser.timezone=Asia/Kolkata -Ddefault_time_zone=Asia/Kolkata
    ports:
      - "8080:8080"
    mem_limit: 3g

volumes:
  localpgdata:
  localosdata:
```

Start it:

```
docker compose -f docker-compose.local.yml up -d
docker compose -f docker-compose.local.yml logs -f moqui    # watch for "started" / errors
```

Moqui creates the entity schema automatically on first boot (~1–2 min). It will be reachable at **http://localhost:8080** once the log settles.

> If you drop OpenSearch to save RAM, also drop the `elasticsearch_url` line; search-backed screens will be degraded but ordinary screens (dashboards, forms) render fine.

Step 3 — get some data (so you can log in and the screen isn't empty)

A fresh DB is **empty — there's no login yet.** Pick one (never the prod dump):

- **Recommended — a sanitized snapshot from the owner.** The owner produces a scrubbed dump (PII, GSTINs, secrets, and `UserAccount` credentials removed/neutralised) with one or two demo tenants. Restore it: `gunzip -c sanitized.sql.gz | docker compose -f docker-compose.local.yml exec -T db psql -U moqui -d moqui`. This gives realistic multi-tenant data for visual checks, safely.
- **Or — load Moqui's demo dataset** for a from-scratch baseline (gives the standard `john.doe` / `moqui` admin). The exact load invocation depends on the image entrypoint — confirm it in `README.md` or with the owner (Moqui's `load types=seed,seed-initial,demo`), then seed a test org for anything tenant-scoped.

Log in at http://localhost:8080 with the admin from your data (demo data = `john.doe` / `moqui`).

Step 4 — put *your* branch's change into the local container

The image has the components baked in, so to test an edited file you overlay it, the same way the production deploy does — but locally:

One-off check (quick):

```
# copy your edited file into the running local container, then reload
docker compose -f docker-compose.local.yml cp \
  component/zenith-erp-contacts/screen/zenith/contacts/Contacts/ContactsDashboard.xml \
  moqui:/opt/moqui/runtime/component/zenith-erp-
contacts/screen/zenith/contacts/Contacts/ContactsDashboard.xml
docker compose -f docker-compose.local.yml restart moqui      # XML/Groovy reload on restart
```

Iterative work (nicer): add a bind-mount for the component you're editing so your local edits are picked up on each `restart moqui` without copying — e.g. under the `moqui` service:

```
    volumes:
      - ./component/zenith-erp-contacts:/opt/moqui/runtime/component/zenith-erp-contacts
```

Static assets (CSS/JS under `zenithstatic/`) are served as-is and browser-cached. After changing one, bump its `?v=N` in `ZenithApp.xml` (both the `html_stylesheets.add( ... )` and the `<link>` lines) or hard-refresh (Ctrl-Shift-R) — same rule as production.

Step 5 — verify the screen

1. **Both shells.** Open the screen on `/apps` **and** `/qapps` (and `/vapps` if the screen declares it) — they render differently and a render-mode screen that omits one renders **blank** there:
   - `http://localhost:8080/apps/zenith/contacts/Contacts/ContactsDashboard`
   - `http://localhost:8080/qapps/zenith/contacts/Contacts/ContactsDashboard`
2. **Watch the logs** while the page loads: `docker compose -f docker-compose.local.yml logs -f moqui`. A Groovy/FreeMarker error shows here even if the page "mostly" renders — treat any stack trace as a failure.
3. **Check the real content:** the data is right, numbers add up, charts draw, no empty/blank panels, layout holds at narrow widths.
4. **Push on it a little:** switch active organization (org-scoped screens must not leak across tenants), try the date/period controls, submit a form, resize the window.
5. *(optional)* Automate it with the `moqui-verify-screen` Puppeteer recipe pointed at `http://localhost:8080` (plain http, no `--ignore-certificate-errors` needed locally).

If it renders cleanly end-to-end, **that** is the integration check — record it in the PR (a note + screenshot).

Step 6 — before the PR can be merged

1. Rebase/merge the latest `master` into your branch and **resolve any conflicts** (`git fetch origin && git merge origin/master`). A stale branch that conflicts with `master` cannot be merged cleanly.
2. **Re-verify (Steps 4–5) after resolving** — conflict resolution can introduce its own bugs.
3. Push, then in GitHub click **"Ready for review"** to take the PR out of Draft.
4. A reviewer/the owner merges and runs the production deploy (`../SOP-commit-and-deploy.md`).

Tear down

```
docker compose -f docker-compose.local.yml down -v     # -v also removes the local DB/OS volumes
```

Troubleshooting

- **Moqui exits / OOM:** give Docker Desktop more memory, or drop the `opensearch` service and its `elasticsearch_url` line.

- **"password authentication failed":** `entity_ds_password` (moqui) must exactly match `POSTGRES_PASSWORD` (db). If you changed it after first boot, recreate the db volume (`down -v` then `up -d`).
- **Can't log in:** the DB has no users yet — do Step 3.
- **Screen 404s:** check the URL casing/path matches the screen's subscreens path; `/qapps` may redirect to `/apps` for screens that are `/apps`-only (e.g. the POS Terminal).
- **CSS/JS change not showing:** bump `?v=N` in `ZenithApp.xml` or hard-refresh.